Demo v1.0

Veduwa

AI-Powered IT Job Marketplace

TECHNICAL WHITEPAPER

Version 1.0 — April 2026

Author: Mukela Katungu

Status: Live Demo

Classification: Client-Facing

Table of Contents

1. Executive Summary 3
2. System Architecture 4
3. Technology Stack 5
4. Database Schema 7
5. Backend API Layer 9
6. AI Pipeline 11
7. Frontend Architecture 13
8. Authentication & Authorization 15
9. Design System 16
10. Infrastructure & Deployment 17
11. Seed Data & Demo Credentials 18
12. Performance & Security 19

1. Executive Summary

Veduwa is an AI-powered IT job marketplace that automates the hiring pipeline from job posting to candidate screening. The platform uses vector embeddings for semantic matching, Claude AI for real-time candidate screening via streaming chat, and a modern full-stack architecture built for production scale.

12	15	5	5
FRONTEND ROUTES	API ENDPOINTS	DATABASE TABLES	AI SERVICES

Live Deployment

SERVICE	URL	STATUS
Frontend	`frontend-vert-tau-84.vercel.app`	Live
Backend API	`veduwa-api.onrender.com`	Live
Health Check	`veduwa-api.onrender.com/health`	OK
Source Code	`github.com/Mukela12/veduwa-demo`	Public

Core Value Proposition

Employers post jobs → AI automatically extracts skills and requirements → Candidates are matched and ranked using vector similarity → Claude AI conducts structured pre-screening interviews → Employers receive plain-English hiring recommendations.

2. System Architecture

Veduwa follows a decoupled architecture with a Next.js frontend deployed on Vercel's edge network, a FastAPI backend on Render, and Supabase providing both PostgreSQL and authentication services.

```
                    ┌──────────────────┐
                    │   Vercel (Edge)   │
                    │   Next.js 14 SSR  │
                    └──────────────────┘
                             │
                             │ HTTPS / REST
                             ▼
┌──────────────┐   ┌──────────────────┐   ┌──────────────────┐
│   Supabase   │◄──►│   Render.com     │◄──►│   Claude AI      │
│   Auth (JWT) │   │   FastAPI 0.115  │   │   (Anthropic)    │
│   PostgreSQL │   │   Uvicorn ASGI   │   │   Haiku + Sonnet │
└──────────────┘   └──────────────────┘   └──────────────────┘
       ▲                     │
       │                     │ asyncpg
       │                     ▼
       │            ┌──────────────────┐
       └────────────│   PostgreSQL     │
                    │   + pgvector     │
                    │   5 tables       │
                    └──────────────────┘
```

Communication Patterns

FLOW	PROTOCOL	AUTH
Client → Vercel	HTTPS (SSR + CSR)	None
Client → Supabase	HTTPS (REST)	Anon key
Client → FastAPI	HTTPS (REST + SSE)	Bearer JWT
FastAPI → Supabase	HTTPS (Auth API)	Service key
FastAPI → PostgreSQL	asyncpg (TCP)	Connection string
FastAPI → Claude	HTTPS (Streaming)	API key

Port Configuration

SERVICE	PORT	ENVIRONMENT
Next.js Dev	3099	Local
FastAPI Dev	8000	Local
PostgreSQL	5432	Supabase Cloud
Redis	6379	Railway (optional)

3. Technology Stack

Backend

CATEGORY	TECHNOLOGY	VERSION	PURPOSE
Framework	FastAPI	0.115.0	Async REST API
ORM	SQLAlchemy	2.0.36	Async database ops
Migrations	Alembic	1.14.0	Schema versioning
Validation	Pydantic	2.10.0	Request/response schemas
Auth	python-jose	3.3.0	JWT decoding
AI	Anthropic SDK	0.42.0	Claude API integration
Task Queue	Celery	5.4.0	Background processing
Cache/Broker	Redis	5.2.0	Celery broker
Server	Uvicorn	0.34.0	ASGI server
File Storage	Cloudinary	1.41.0	Resume uploads
Email	Resend	2.0.0	Transactional email

Frontend

CATEGORY	TECHNOLOGY	VERSION	PURPOSE
Framework	Next.js	14.2.35	React meta-framework
Language	TypeScript	5.x	Type safety
Styling	Tailwind CSS	4.x	Utility-first CSS
Animation	Framer Motion	12.38.0	Page transitions, gestures
Icons	Lottie React	2.4.1	25 animated LordIcons
Login	Rive Canvas	latest	Interactive Rive animation
Charts	Recharts	3.8.1	Dashboard visualizations
Auth	Supabase SSR	0.10.0	Auth state management
Theme	next-themes	0.4.6	Dark/light mode toggle
UI Primitives	Radix UI	various	Accessible components

Infrastructure

SERVICE	PROVIDER	REGION	PLAN
Database + Auth	Supabase	US East (Virginia)	Free
Backend Hosting	Render.com	Ohio	Free
Frontend Hosting	Vercel	Edge (global)	Hobby
AI Provider	Anthropic	US	Pay-per-use
Redis (optional)	Railway	US East	Free

4. Database Schema

PostgreSQL with pgvector extension for semantic vector similarity search. 5 tables managed via Alembic migrations.

Entity Relationship

```
users ──┬── jobs (employer_id FK)
        │
        └── candidates (user_id FK, unique)
                │
                └── applications (candidate_id, job_id)  UNIQUE(job_id, candidate_id)
                        │
                        └── screenings (application_id FK)
```

Table: users

COLUMN	TYPE	CONSTRAINTS
`id`	UUID	PK, default gen_random_uuid()
`email`	VARCHAR(255)	UNIQUE, NOT NULL, INDEXED
`full_name`	VARCHAR(255)	NOT NULL
`role`	VARCHAR(20)	CHECK IN ('employer', 'candidate')
`avatar_url`	VARCHAR	NULLABLE
`company_name`	VARCHAR(255)	NULLABLE
`created_at`	TIMESTAMPTZ	DEFAULT NOW()
`updated_at`	TIMESTAMPTZ	DEFAULT NOW()

Table: jobs

COLUMN	TYPE	CONSTRAINTS
`id`	UUID	PK
`employer_id`	UUID	FK → users.id, INDEXED
`title`	VARCHAR(255)	NOT NULL, INDEXED
`company`	VARCHAR(255)	NOT NULL, INDEXED
`description`	TEXT	NOT NULL
`location`	VARCHAR(255)	NULLABLE
`job_type`	VARCHAR(20)	CHECK ('full_time','contract','remote')
`salary_min / max`	INTEGER	NULLABLE
`seniority`	VARCHAR(20)	CHECK ('junior','mid','senior','lead')
`skills`	JSONB	DEFAULT '[]'
`parsed_data`	JSONB	NULLABLE (AI-generated)
`embedding`	VECTOR(384)	NULLABLE (semantic vector)
`status`	VARCHAR(20)	DEFAULT 'active', INDEXED

Table: candidates

COLUMN	TYPE	CONSTRAINTS
`id`	UUID	PK
`user_id`	UUID	FK → users.id, UNIQUE
`resume_url`	VARCHAR	NULLABLE
`parsed_resume`	JSONB	NULLABLE (AI-generated)
`skills`	JSONB	DEFAULT '[]'
`experience_years`	INTEGER	NULLABLE
`embedding`	VECTOR(384)	NULLABLE

Table: applications

COLUMN	TYPE	CONSTRAINTS
`id`	UUID	PK
`job_id`	UUID	FK → jobs.id
`candidate_id`	UUID	FK → candidates.id
`match_score`	FLOAT	NULLABLE (0-100)
`skills_overlap`	JSONB	NULLABLE
`status`	VARCHAR(20)	DEFAULT 'pending'

Unique constraint: (job_id, candidate_id)

Table: screenings

COLUMN	TYPE	CONSTRAINTS
`id`	UUID	PK
`application_id`	UUID	FK → applications.id
`messages`	JSONB	DEFAULT '[]'
`summary`	TEXT	NULLABLE
`outcome`	VARCHAR(20)	DEFAULT 'pending' (pass/fail/pending)
`started_at`	TIMESTAMPTZ	DEFAULT NOW()
`completed_at`	TIMESTAMPTZ	NULLABLE

5. Backend API Layer

Route Modules

MODULE	PREFIX	ENDPOINTS	AUTH
auth	`/api/auth`	1	Public
jobs	`/api/jobs`	5	Mixed
candidates	`/api/candidates`	3	Required
applications	`/api/applications`	3	Required
screening	`/api/screening`	4	Required
dashboard	`/api/dashboard`	2	Employer only

Complete Endpoint Specification

METHOD	PATH	AUTH	DESCRIPTION
GET	`/health`	—	Health check
POST	`/api/auth/verify`	—	Verify Supabase JWT
GET	`/api/jobs`	—	List jobs (paginated, filtered)
POST	`/api/jobs`	Employer	Create job + AI parse
GET	`/api/jobs/{id}`	—	Job detail
PATCH	`/api/jobs/{id}`	Owner	Update job
DEL	`/api/jobs/{id}`	Owner	Soft-delete (close)
GET	`/api/candidates`	Auth	List candidates
GET	`/api/candidates/{id}`	Auth	Candidate profile
POST	`/api/candidates/{id}/resume`	Owner	Upload + AI parse
POST	`/api/applications`	Candidate	Apply to job
GET	`/api/applications/{id}/matches`	Auth	Match results
PATCH	`/api/applications/{id}/status`	Employer	Update status
POST	`/api/screening/{id}/start`	Employer	Start AI screening
GET	`/api/screening/{id}/stream`	Auth	SSE streaming chat
POST	`/api/screening/{id}/message`	Auth	Send message
POST	`/api/screening/{id}/summarize`	Employer	Generate summary
GET	`/api/dashboard/stats`	Employer	KPI metrics
GET	`/api/dashboard/pipeline`	Employer	Candidate funnel

Pagination Response Format

```
{
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "per_page": 20,
    "total": 156,
    "total_pages": 8
  }
}
```

6. AI Pipeline

All AI operations use the Anthropic Claude API exclusively. No OpenAI dependency. The pipeline processes jobs and candidates through 5 stages.

```
Job Description ──► JD Parser (Haiku) ──► Structured JSON
                                                  │
Resume PDF ──────────► Resume Parser (Haiku) ────►│
                                                  ▼
                                        Embedding Generator
                                        (384-dim vectors)
                                                  │
                                                  ▼
                                         Match Scorer
                                      (cosine + skills)
                                                  │
                                                  ▼
                                       AI Screening Chat
                                      (Sonnet streaming)
                                                  │
                                                  ▼
                                      Summary Generator
                                         (Sonnet)
```

AI Services

SERVICE	MODEL	PURPOSE	OUTPUT
JD Parser	`claude-haiku-4-5`	Extract skills, seniority, domain	JSON
Resume Parser	`claude-haiku-4-5`	Extract skills, experience, education	JSON
Embeddings	Hash-based (384d)	Semantic vector generation	float[384]
Match Scorer	NumPy	Cosine similarity + skills overlap	0–100 score
Screening Chat	`claude-sonnet-4`	Real-time interview	SSE stream
Summarizer	`claude-sonnet-4`	Employer assessment	Text

Match Score Formula

match_score = (cosine_similarity × 0.4 + skills_overlap_ratio × 0.6) × 100

- **Cosine similarity (40%)** — Semantic similarity between job and candidate embedding vectors
- **Skills overlap (60%)** — Ratio of matched skills to required skills, case-insensitive

Celery Background Tasks

TASK	TRIGGER	PROCESSING
`parse_job_description`	Job creation	Claude Haiku → structured JSON
`parse_resume`	Resume upload	Claude Haiku → skills extraction
`compute_embeddings`	After parsing	384-dim vector generation
`compute_match_scores`	After embeddings	Batch cosine + overlap scoring

7. Frontend Architecture

Route Map

ROUTE	PAGE	AUTH	DESCRIPTION
`/`	Landing	—	Marketing hero, stats, features
`/auth/login`	Login	—	Rive animation + demo cards + OAuth
`/auth/signup`	Signup	—	Role selection + registration
`/dashboard`	Dashboard	Employer	Stats + pipeline chart + table
`/jobs`	Job Board	—	Filter + search + cards
`/jobs/[id]`	Job Detail	—	Full job view
`/dashboard/post`	Post Job	Employer	4-step form wizard
`/screening`	AI Screening	Employer	70/30 split chat + profile
`/candidates`	Candidates	Auth	Grid + status filters
`/candidates/[id]`	Profile	Auth	Tabs + timeline + documents

Key UI Components

COMPONENT	PURPOSE	ANIMATION
LordIcon (25 icons)	Micro-interaction icons	Lottie hover/click/loop
MatchScoreRing	SVG donut 0–100	CSS stroke-dashoffset 1s
Dock	Mobile navigation	Framer Motion spring physics
ThemeToggle	Sun/moon dark mode	CSS cubic-bezier 0.5s
AnimationPanel	Rive login illustration	Rive state machine
StatCard	KPI with delta trend	None (data-focused)
JobCard	Rich job listing	Transform translateY
CandidateCard	Photo + skills + score	Hover elevation

Data Flow Pattern

Every page follows a **graceful degradation pattern**: fetch from the live API on mount, fall back to mock data if the API is unreachable. Pages display a `(demo data)` label when using mock fallback. This ensures the demo always renders beautifully regardless of backend state.

8. Authentication & Authorization

Auth Flow

```
1. User signs in via Supabase Auth (email/password or Google OAuth)
2. Supabase returns JWT access token
3. Frontend stores token in Supabase session (httpOnly cookie)
4. API requests include: Authorization: Bearer <token>
5. Backend verifies token via Supabase Auth API
6. User record auto-created in DB on first login
7. Role extracted from user_metadata.role ('employer' | 'candidate')
```

Role-Based Access Control

RESOURCE	EMPLOYER	CANDIDATE	PUBLIC
View jobs	✓	✓	✓
Post jobs	✓	—	—
View candidates	✓	—	—
Apply to jobs	—	✓	—
Start screening	✓	—	—
View dashboard	✓	—	—
Upload resume	—	✓	—
Generate summary	✓	—	—

9. Design System — Clarity Theme

Inspired by Linear.app. Clean, minimal, Swiss typography, maximum data-ink ratio. Full dark mode support.

Light Mode Palette

TOKEN	VALUE	USAGE
background	`#FAFAF8`	Page bg
foreground	`#1C1C1A`	Text
primary	`#3B5BDB`	Buttons, links
surface	`#FFFFFF`	Cards
border	`#E6E4DF`	Borders
success	`#40C057`	Positive
warning	`#FAB005`	Caution
error	`#FA5252`	Errors

Dark Mode Palette

TOKEN	VALUE	USAGE
background	`#0C0C0E`	Page bg
foreground	`#EDEDEB`	Text
primary	`#4C6EF5`	Buttons, links
surface	`#161618`	Cards
border	`#2A2A2E`	Borders
success	`#4ADE80`	Positive
warning	`#FBBF24`	Caution
error	`#F87171`	Errors

Typography

FONT	WEIGHT	USAGE
Satoshi (fontshare)	600–700	Headings, branding
Inter (Google Fonts)	400–500	Body text, labels
JetBrains Mono	400	Data, scores, code

Responsive Strategy

Desktop (768px+)

- Fixed sidebar navigation (240px)
- Full header with search + Cmd+K
- Theme toggle in sidebar footer
- Multi-column layouts

Mobile (<768px)

- macOS-style Dock (Framer Motion spring)
- No sidebar — dock replaces nav
- Compact header with logo + search icon
- Single-column stacked layouts

10. Infrastructure & Deployment

Vercel (Frontend)

- Framework: Next.js
- Build: `npm run build`
- Region: Edge (global)
- Auto-deploy: GitHub push

Render (Backend)

- Runtime: Docker
- Base: Python 3.12-slim
- Region: Ohio
- Health: `/health`

Supabase (Data)

- PostgreSQL + pgvector
- Auth (email + OAuth)
- Region: US East
- Connection pooler

Environment Variables

Frontend (Vercel)

VARIABLE	PURPOSE
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase anon key
`NEXT_PUBLIC_API_URL`	Backend API URL

Backend (Render)

VARIABLE	PURPOSE
`DATABASE_URL`	Postgres connection
`SUPABASE_URL`	Auth verification
`ANTHROPIC_API_KEY`	Claude AI access
`CORS_ORIGINS`	Allowed origins
`DEMO_AUTH`	Demo mode toggle

Veduwa DEMO DATA

11. Seed Data & Demo Credentials

Demo Login

Email: `demo@veduwa.com` **Password:** `demo123` **Role:** Employer

Seeded Data Summary

10	6	8	15	1
USERS	JOBS	CANDIDATES	APPLICATIONS	SCREENING

Sample Jobs

TITLE	COMPANY	SALARY	TOP SKILLS
Senior Full-Stack Engineer	TechFlow AI	$150–200K	React, Node.js, TypeScript
ML Engineer — NLP Focus	DataMind Labs	$140–180K	Python, PyTorch, Transformers
Frontend Developer	Nexus Design	$120–160K	React, Tailwind, Next.js
DevOps Engineer	CloudScale Inc	$130–170K	Kubernetes, Terraform, AWS
Backend Engineer — Python	FinAI Solutions	$100–140K	FastAPI, PostgreSQL, Redis
AI Research Intern	DeepLogic AI	$60–80K	TensorFlow, NLP, Python

12. Performance & Security

Security Measures

- JWT-based auth via Supabase
- Role-based access control (RBAC)
- CORS whitelist configuration
- No secrets in client-side code
- Environment variables for credentials
- SQL injection prevention (SQLAlchemy ORM)
- Input validation (Pydantic v2)
- HTTPS enforced on all endpoints

Performance Features

- Async database operations (asyncpg)
- Connection pooling (Supabase pooler)
- Static page generation (Next.js SSG)
- Lazy-loaded API data with fallback
- GPU-accelerated animations (Framer)
- Image optimization (next/image)
- Code splitting per route
- Edge deployment (Vercel CDN)

Veduwa Technical Whitepaper — v1.0 — April 2026 — Mukela Katungu

For questions or a walkthrough, contact via Upwork or GitHub.